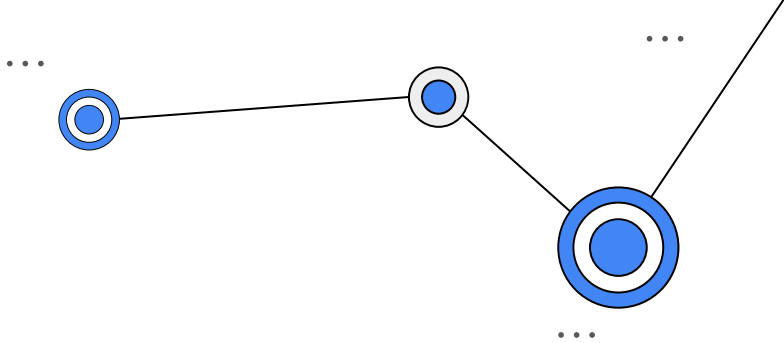


Machine Learning Operations

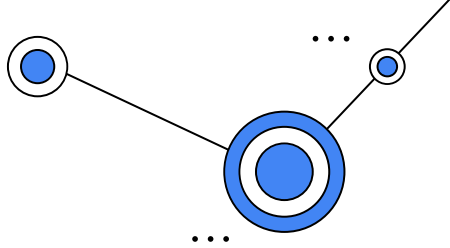
MLOps: Introduction to Machine Learning

Edith Villegas
edith.villegas@areasciencepark.it

Introduction to ML

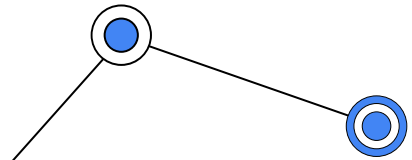
MLOps: Introduction to ML

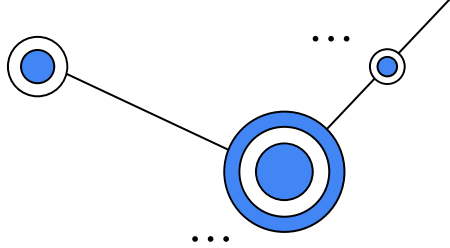
UNIVERSITÀ
DEGLI STUDI
DI TRIESTE



Our Agenda:

1. What is Machine Learning?
2. Supervised vs Unsupervised learning
3. Regression vs Classification
4. Regression
5. Machine Learning Workflow
6. Classification
7. Some Classification Algorithms (knn, decision trees, logistic regression, SVM)
8. Examples





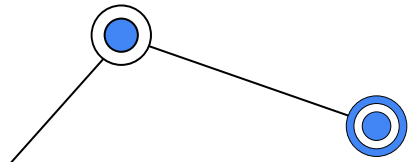
What is Machine Learning?

- Algorithms that learn from experience (data).
- Allows computers to learn without being explicitly programmed.

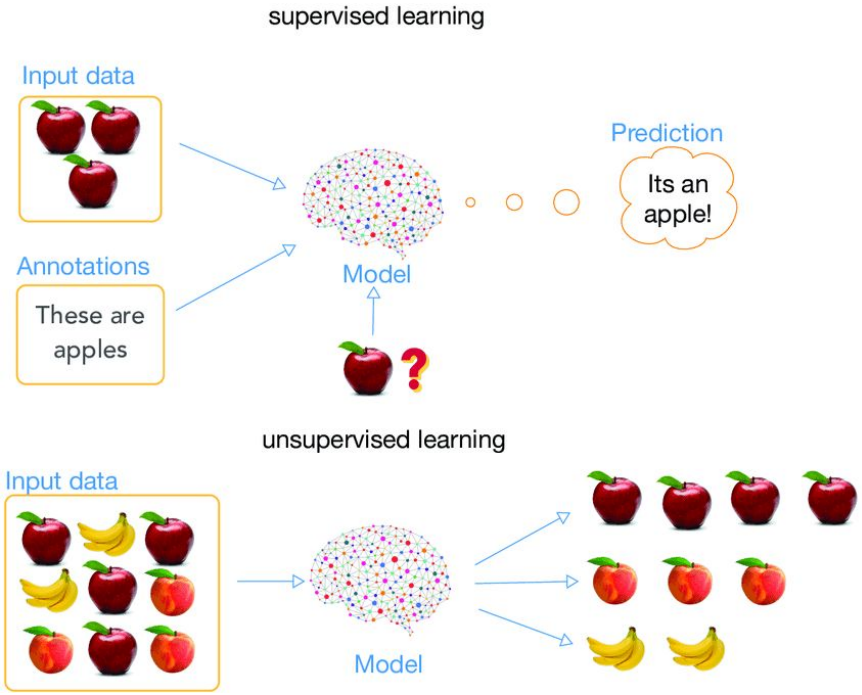
Examples:

- Classifying if a tumor is benign or malignant based on its shape and size.
- Predicting housing prices based on number of rooms, area of the property, location.
- Decide if an email is spam or not based on its contents.

The target value (benign/malignant tumor) is usually called *label*.
The values we give to the algorithm as input are called *features*.

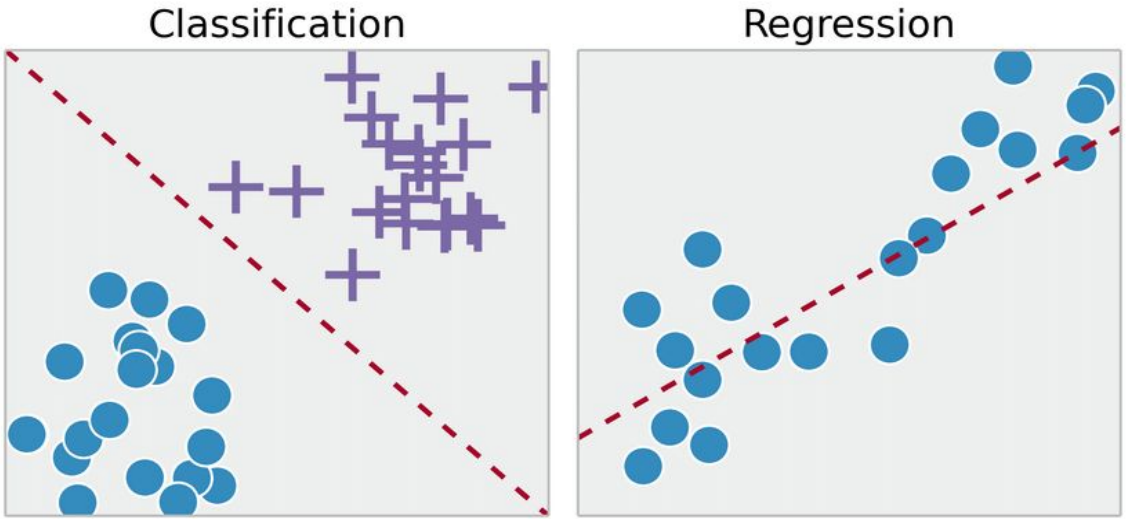


Unsupervised vs Supervised Machine Learning



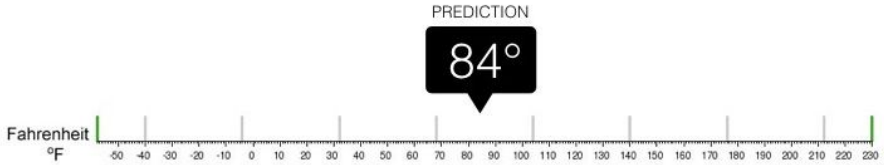
Supervised Learning

- Learning from data with labels



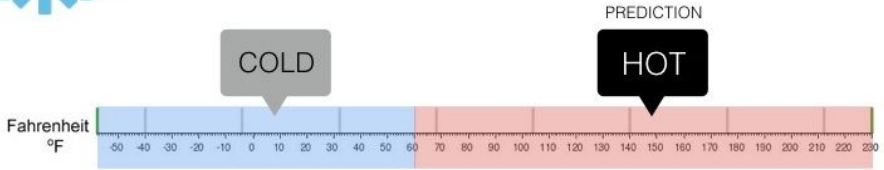
Regression

What is the temperature going to be tomorrow?

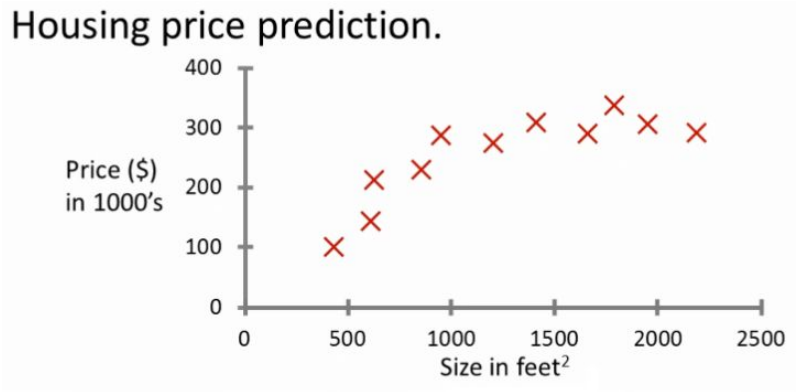


Classification

Will it be Cold or Hot tomorrow?



Supervised Learning



Input (dependent) variables:

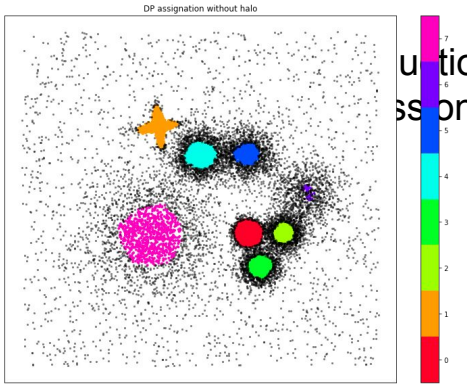
- Number of rooms
- Age of the house
- Number of bathrooms
- Average room area
- Total area
- Neighbourhood

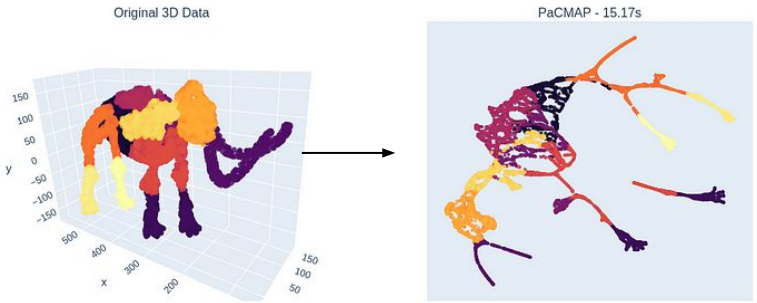
Unsupervised Learning

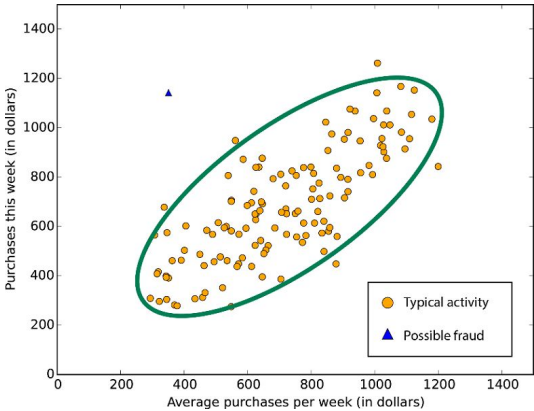
Learning from unlabeled data

- Clustering

(ex. Customer segmentation)





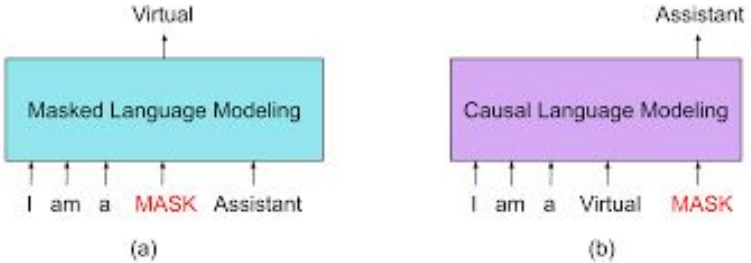


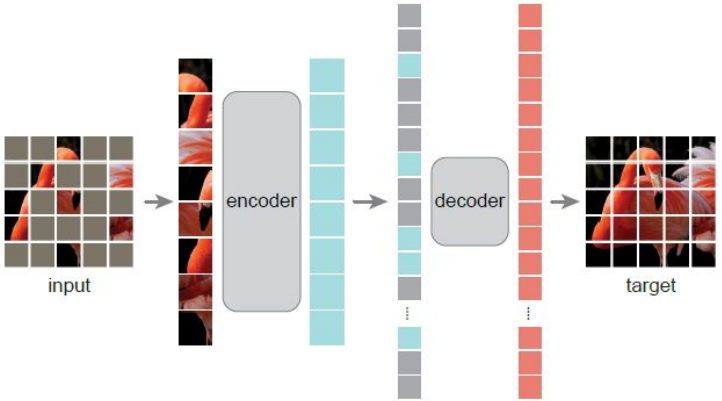
Self-supervised Learning

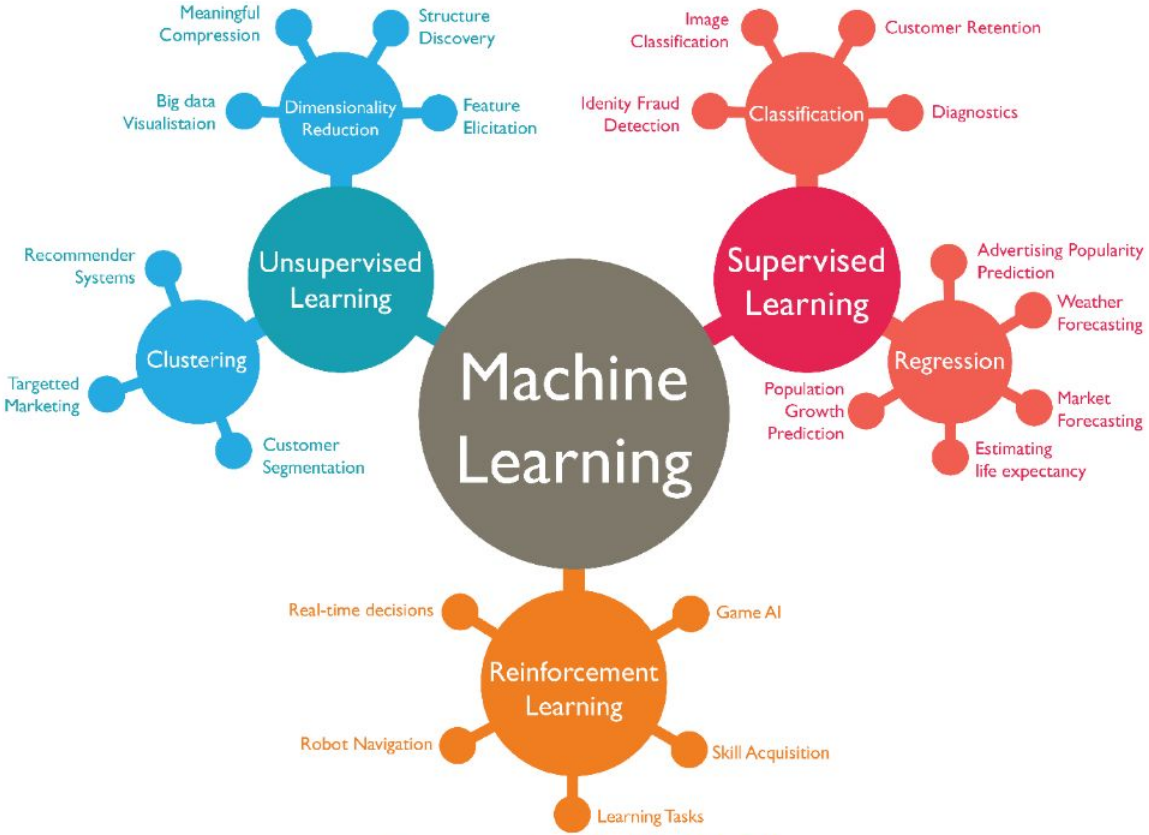
Generating labels from the data itself

- Language

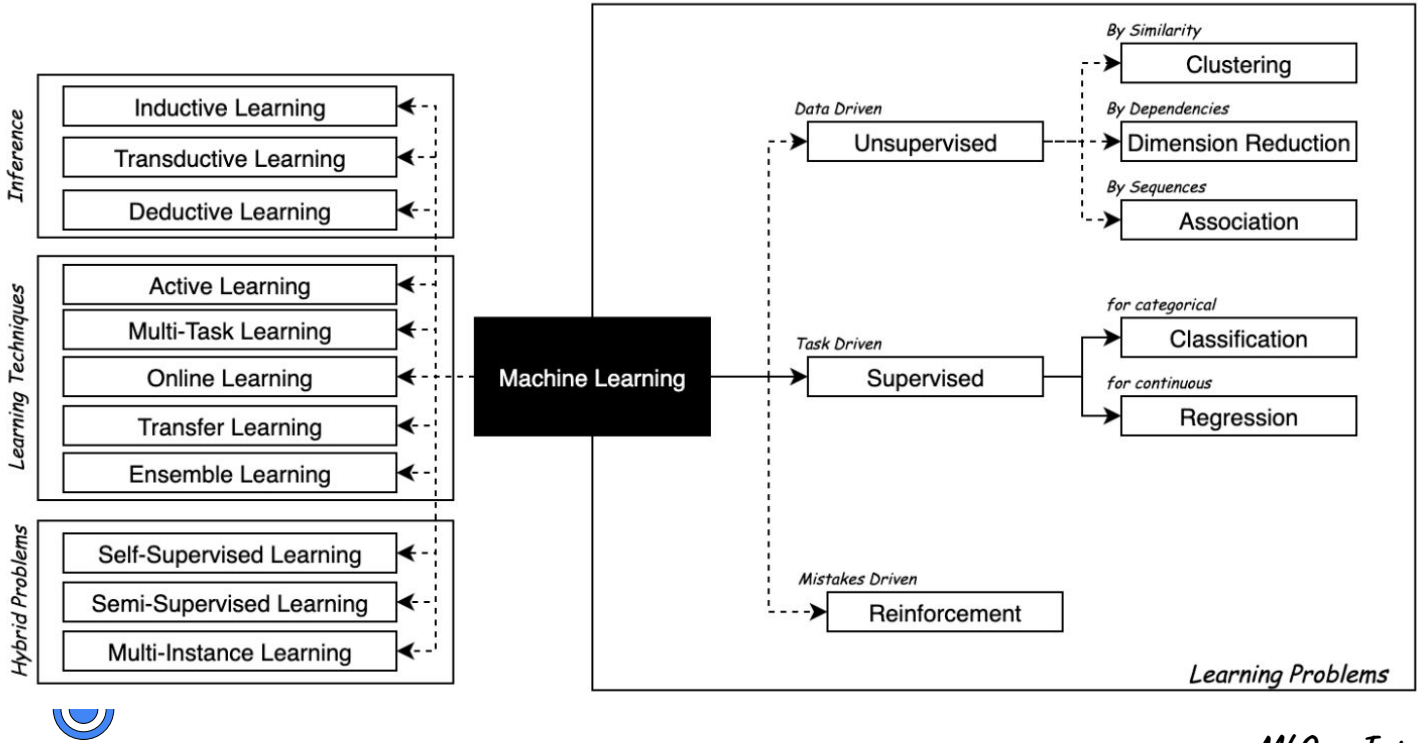
- Vision






Machine Learning
Unsupervised Learning
Dimensionality Reduction
Meaningful Compression
Structure Discovery
Big data Visualistaion
Feature Elicitation
Clustering
Recommender Systems
Targetted Marketing
Customer Segmentation
Supervised Learning
Classification
Image Classification
Customer Retention
Idenity Fraud Detection
Diagnostics
Regression
Advertising Popularity Prediction
Weather Forecasting
Market Forecasting
Estimating life expectancy
Population Growth Prediction
Reinforcement Learning
Real-time decisions
Game AI
Robot Navigation
Skill Acquisition
Learning Tasks

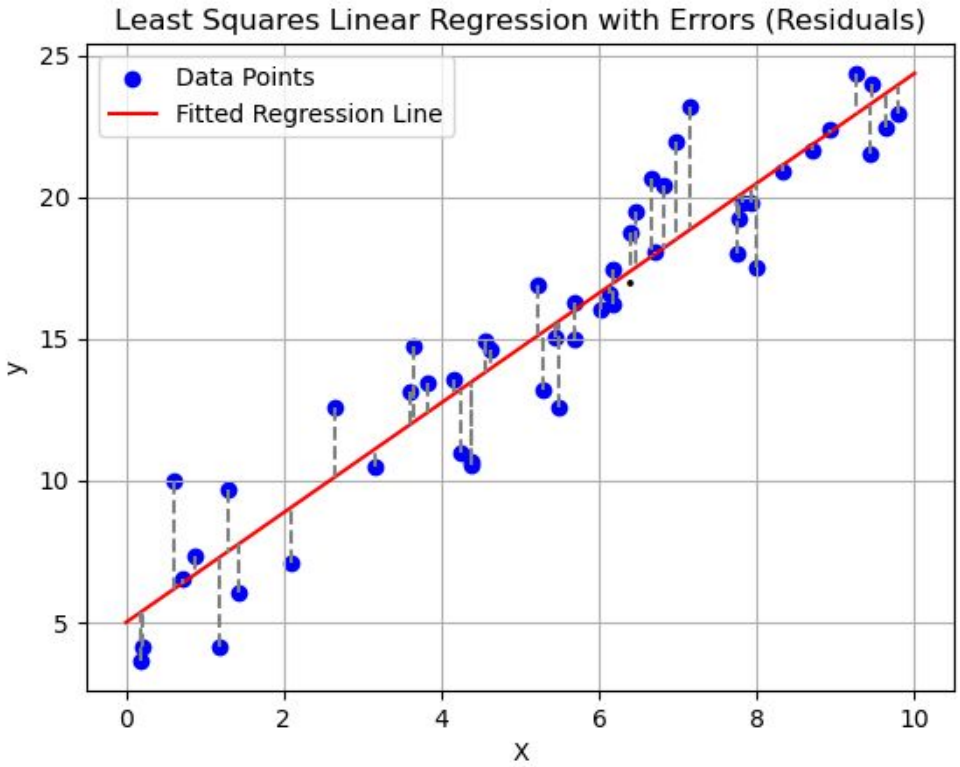
Machine Learning



Regression

Regression

$\hat{y} = ax+b$



To polynomial, exponential, etc.

Some more examples

- **Stock Price Prediction**
  - Predict the future price of a stock based on historical data like past prices, trading volume, and other financial indicators.

- **Medical Risk Prediction (ex. Heart Disease)**
  - Predict the risk of disease or the progression of a condition as a continuous risk score based on factors like age, cholesterol level, blood pressure, lifestyle variables, and family history.

Evaluation

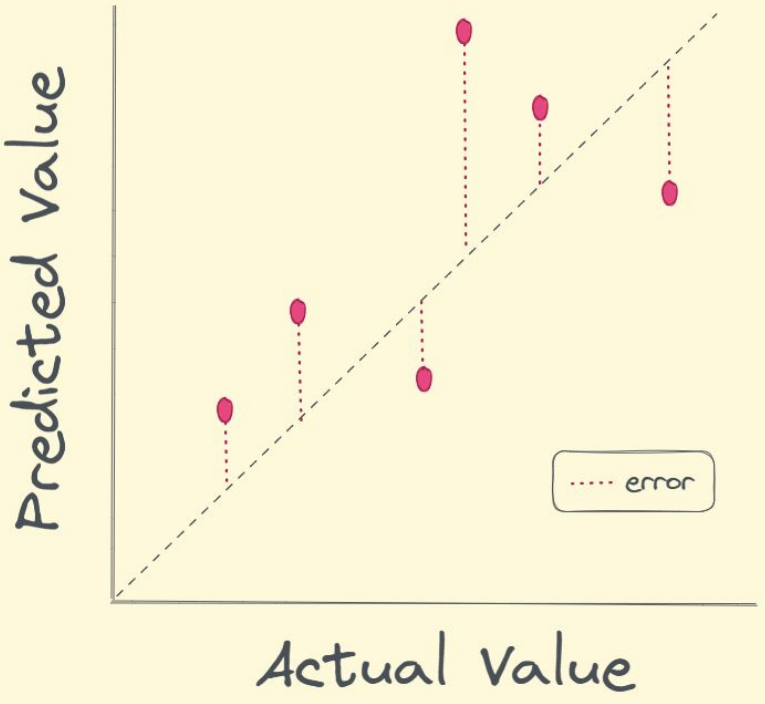
$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

$$R^2 = 1 - \frac{SSR}{SST} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

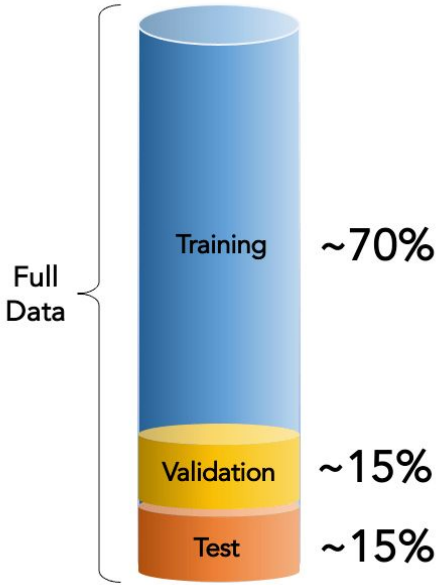
$$MAPE = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$



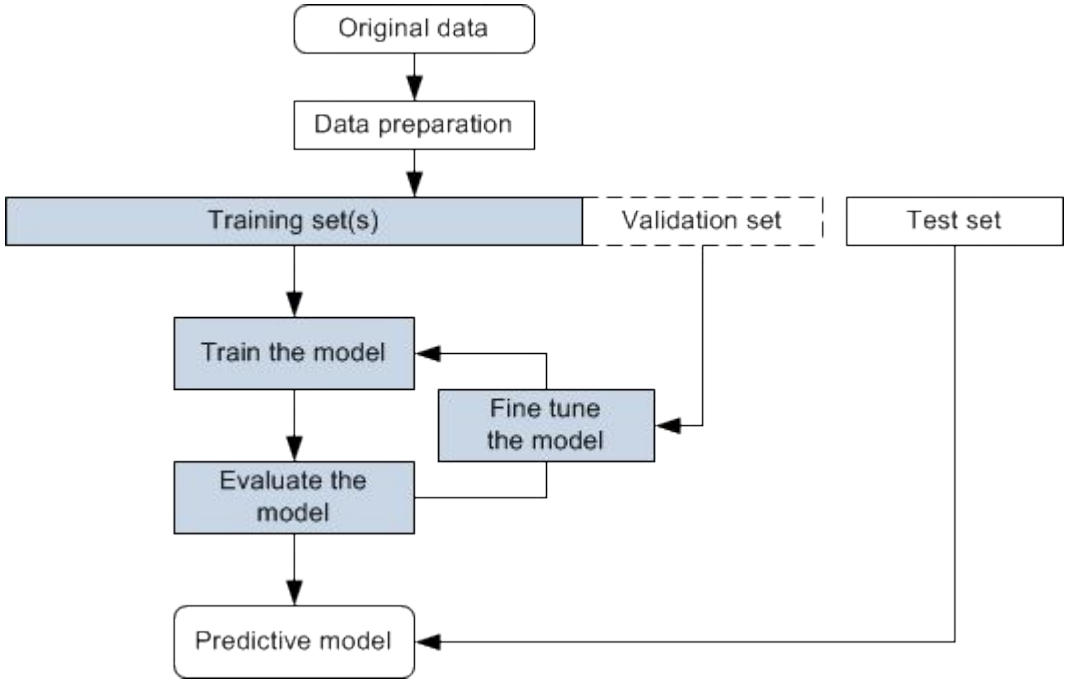
Evaluation

- Test data should not be “seen” by the model during training/fitting.
- Validation data is used for model selection.

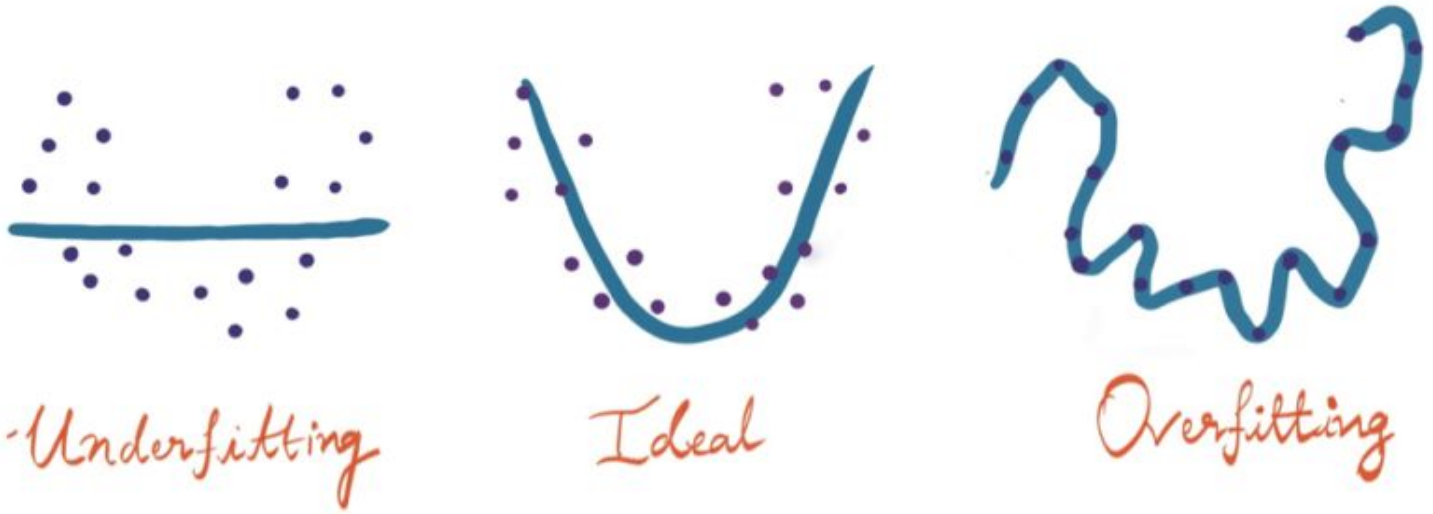
- Test dataset should be closest to data coming from our real use case. It quantifies the quality/predicted performance of the model.



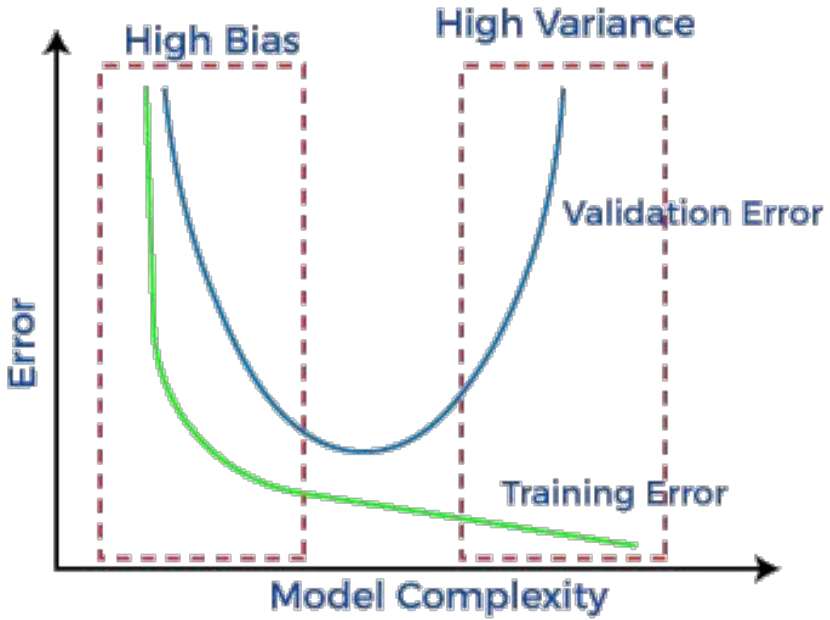
Basic Workflow in Machine Learning

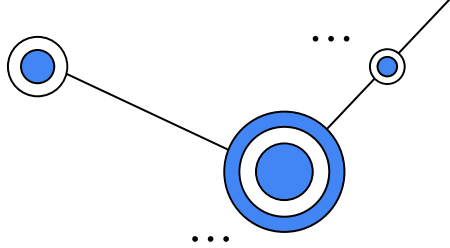


Bias vs Variance Trade-off

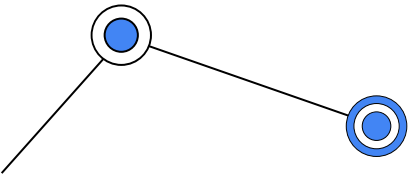


Bias vs Variance Trade-off

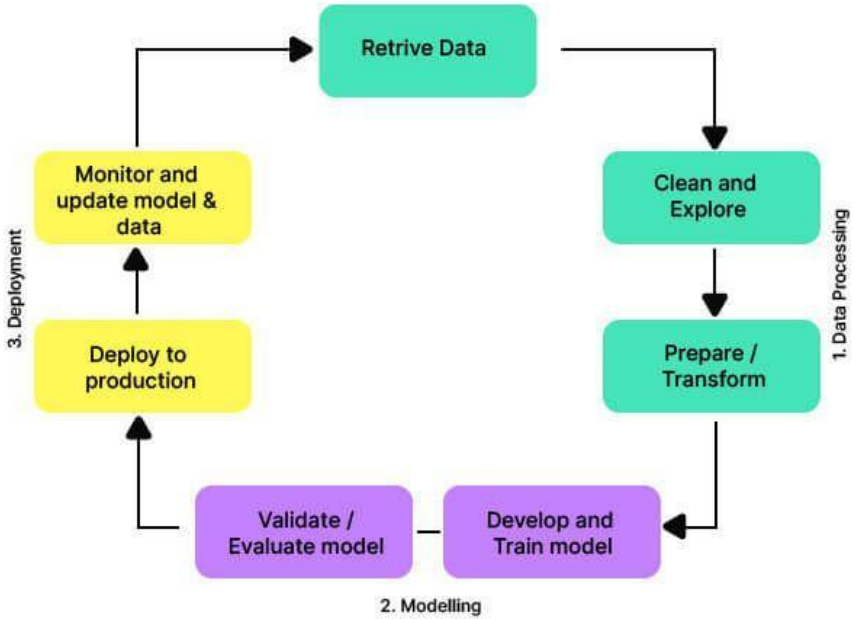




Regression Examples

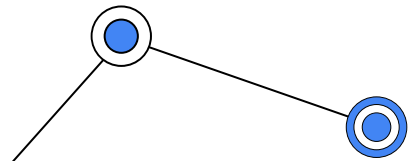


Machine Learning Cycle



Data Preparation

- Data cleaning
- Transformation
- Encoding
- Feature selection
- Handling correlations
- Scaling & Normalization

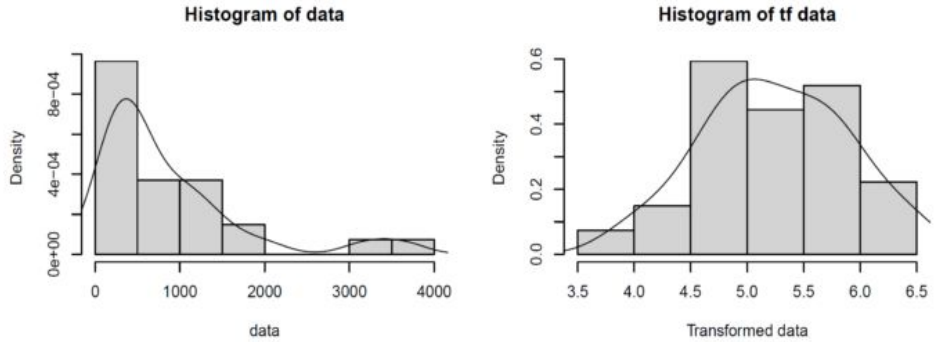


Data Preparation

Data Cleaning: This involves identifying and handling missing values, outliers, and inconsistencies in your dataset. Missing data can be imputed or removed, outliers can be addressed through techniques like winsorization or transformation, and inconsistent data can be corrected or flagged.

Data Preparation

Data Transformation: Data may require transformations to meet the assumptions of the regression model. For example, you may need to log-transform skewed variables or use Box-Cox transformations to make the data more normally distributed.



Data Preparation

Encoding Categorical Variables: If your dataset includes categorical variables, you'll need to encode them into numerical form. This can be done using **one-hot encoding**, **label encoding**, or other methods, depending on the nature of the data and the requirements of the regression model.

Original Data

Team	Points
A	25
A	12
B	15
B	14
B	19
B	23
C	25
C	29

One-Hot Encoded Data

Team_A	Team_B	Team_C	Points
1	0	0	25
1	0	0	12
0	1	0	15
0	1	0	14
0	1	0	19
0	1	0	23
0	0	1	25
0	0	1	29

Data Preparation

Feature Selection and Engineering: Feature selection techniques help you choose the most important predictors and reduce the dimensionality of the dataset, which can improve model performance and reduce overfitting.

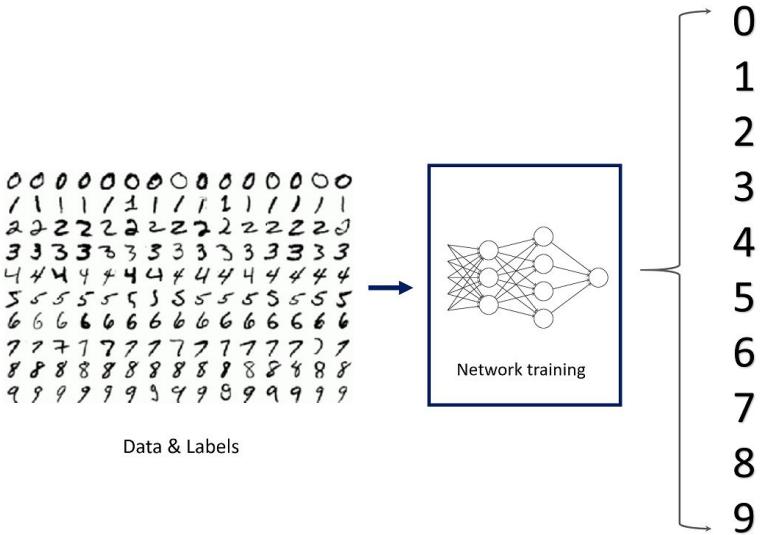
Handling Multicollinearity: Multicollinearity occurs when independent variables in your regression model are **highly correlated with each other**. This can make it challenging to interpret the individual effects of these variables. Techniques like variance inflation factor (VIF) analysis can help identify and mitigate multicollinearity.

Data Scaling and Normalization: This is particularly important when using regression techniques that are sensitive to the scale of variables, such as gradient-based optimization algorithms.

Classification

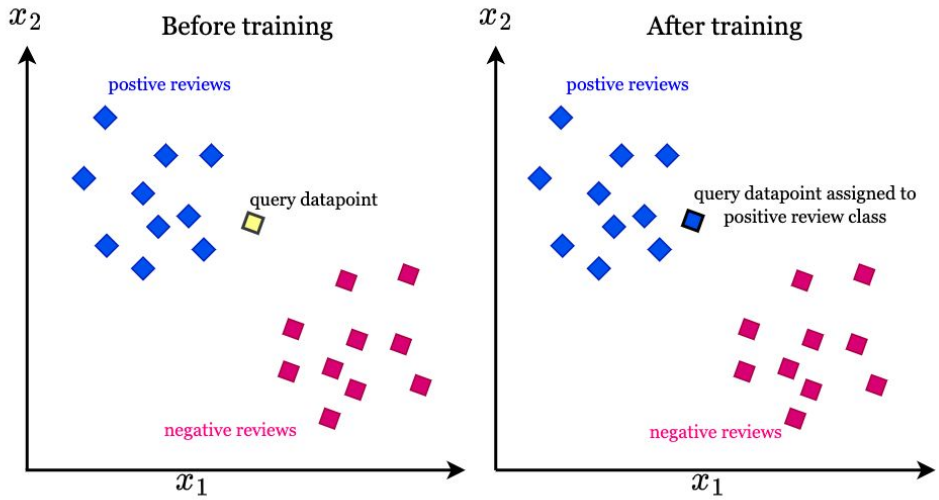
Classification

- Classification is a supervised machine learning method where the model tries to predict the correct label of a given input data.
- Some examples:
  - Spam/not spam emails
  - Positive/negative sentiment in a product review
  - Covid/Pneumonia in chest x-rays
  - Malignant/non-malignant mass in a mammogram



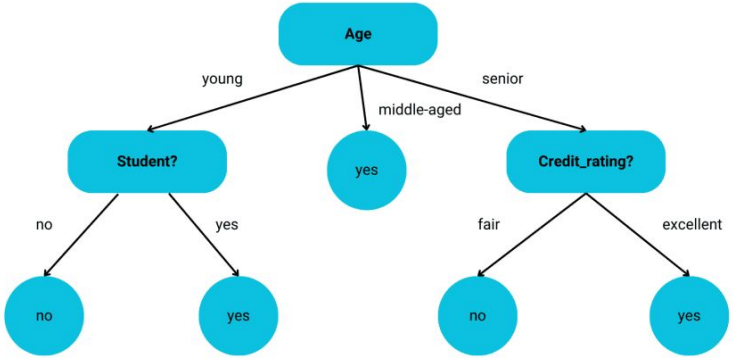
Data & Labels

K Nearest Neighbors

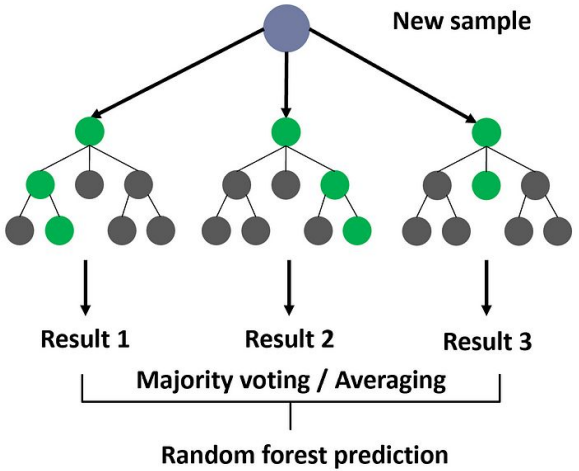


Decision Trees & Random Forests

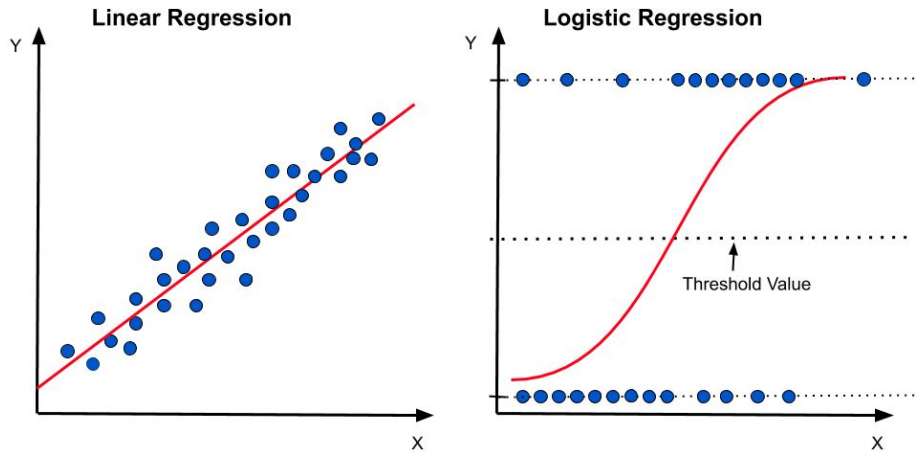
Decision Tree



Random Forest

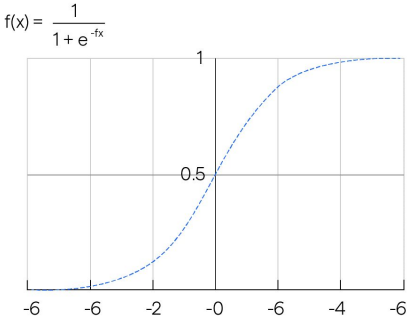


Logistic Regression

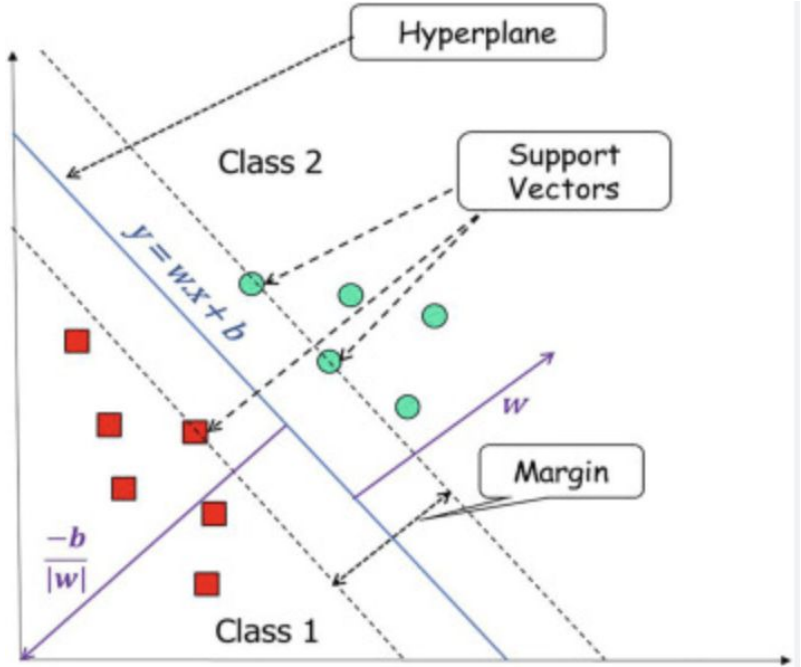


$$\frac{e^{(\beta_0+\beta_1 x)}}{1+e^{(\beta_0+\beta_1 x)}}$$

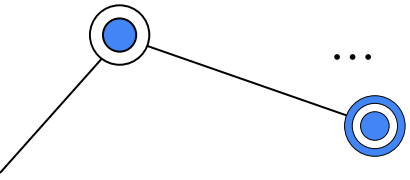
Sigmoid Function

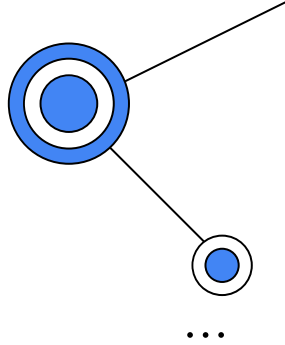


Support Vector Machines



Evaluation



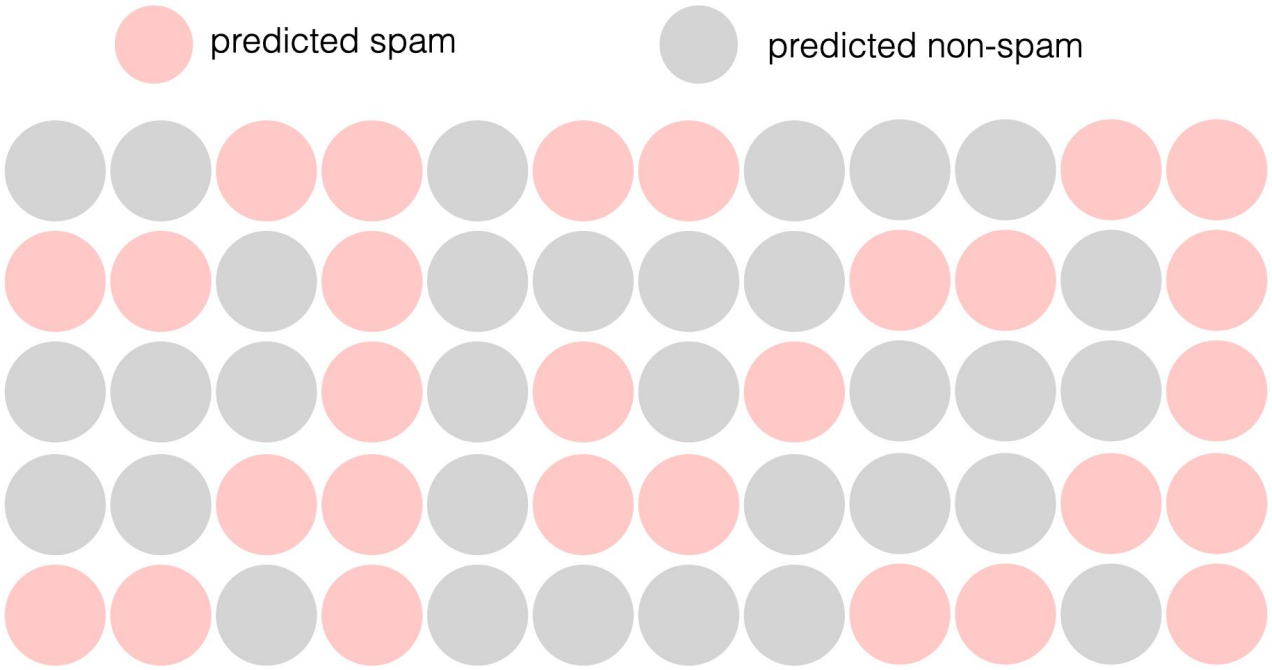


Evaluation:

- **Accuracy** shows how often a classification ML model is correct **overall**.
- **Precision** shows how often an ML model is correct when **predicting the target class.**
- **Recall** shows whether an ML model can find **all objects of the target class**.
- F1 score

UNIVERSITÀ DEGLI STUDI DI TRIESTE

Evaluating Binary Classification

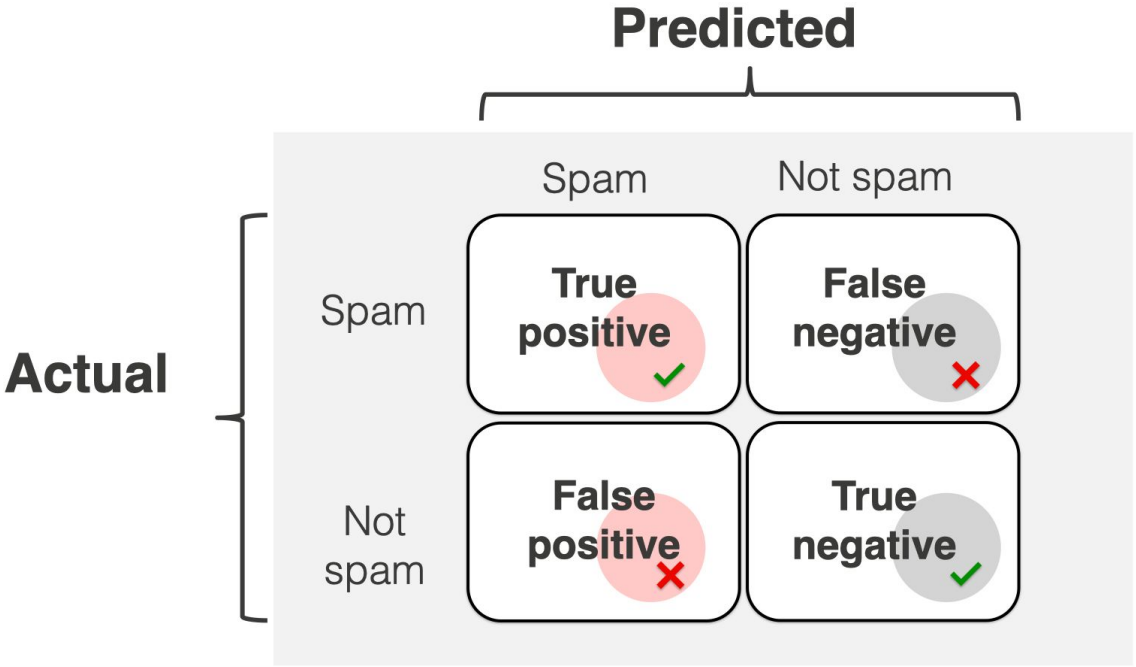


Evaluating Binary Problems

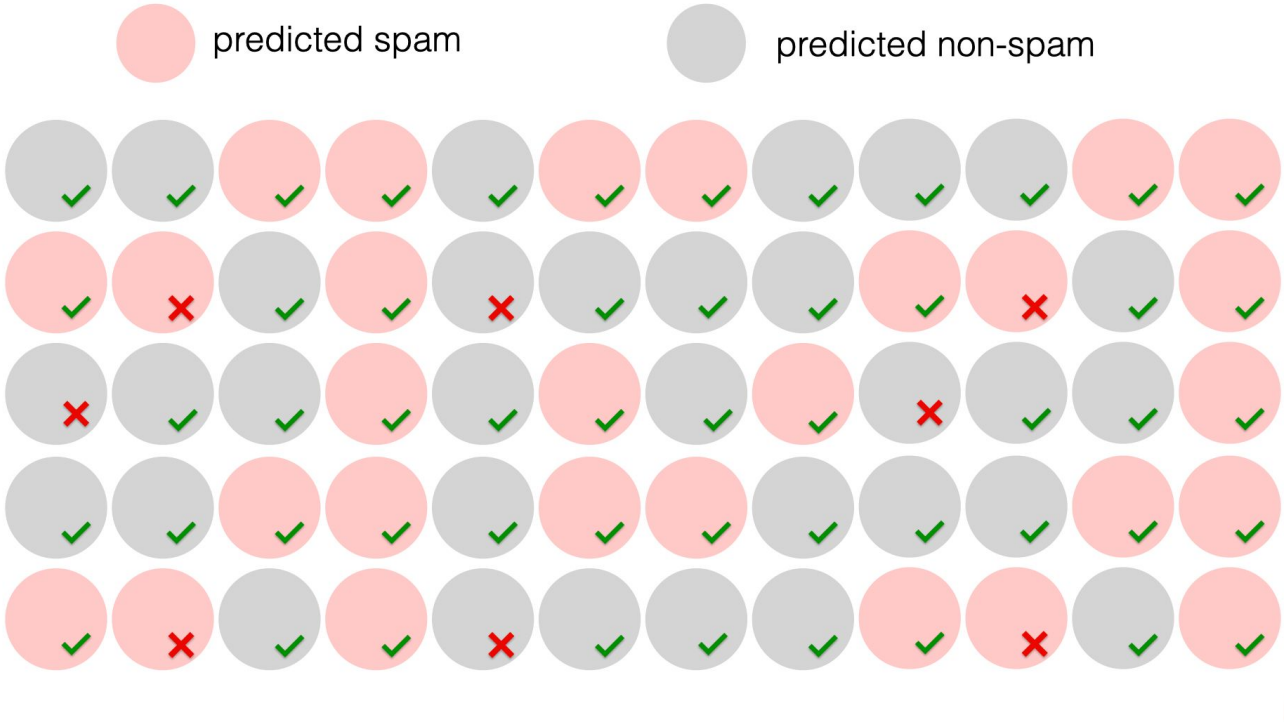
In **binary** classification, we can be "correct" and "wrong" in two different ways:

- **True Positive (TP):** An email that is actually spam and is correctly classified by the model as spam.
- **True Negative (TN):** An email that is actually not spam and is correctly classified by the model as not spam.
- **False Positive (FP):** An email that is actually not spam but is incorrectly classified by the model as spam (a "false alarm").
- **False Negative (FN):** An email that is actually spam but is incorrectly classified by the model as not spam (a "missed spam").

Confusion Matrix:

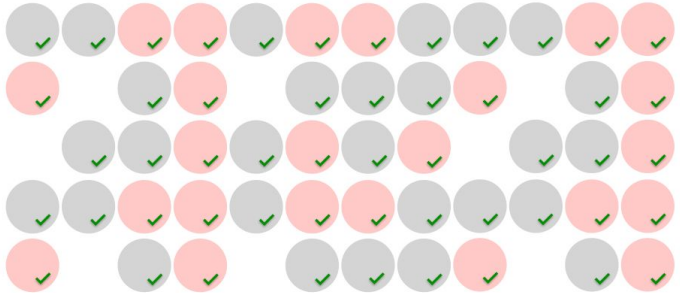


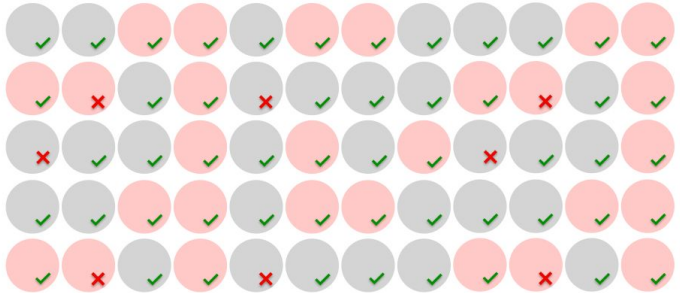
Evaluation:

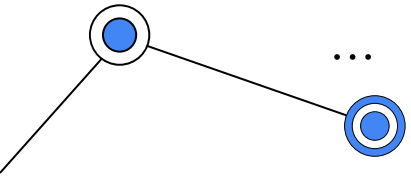


Accuracy

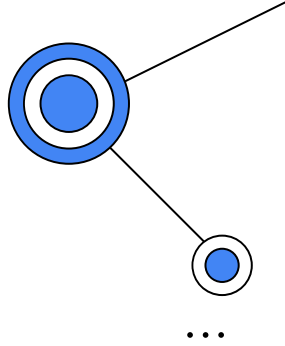
$$\text{Accuracy} = \frac{\text{correct predictions}}{\text{all predictions}} = \frac{52}{60}$$







Evaluation (Accuracy):



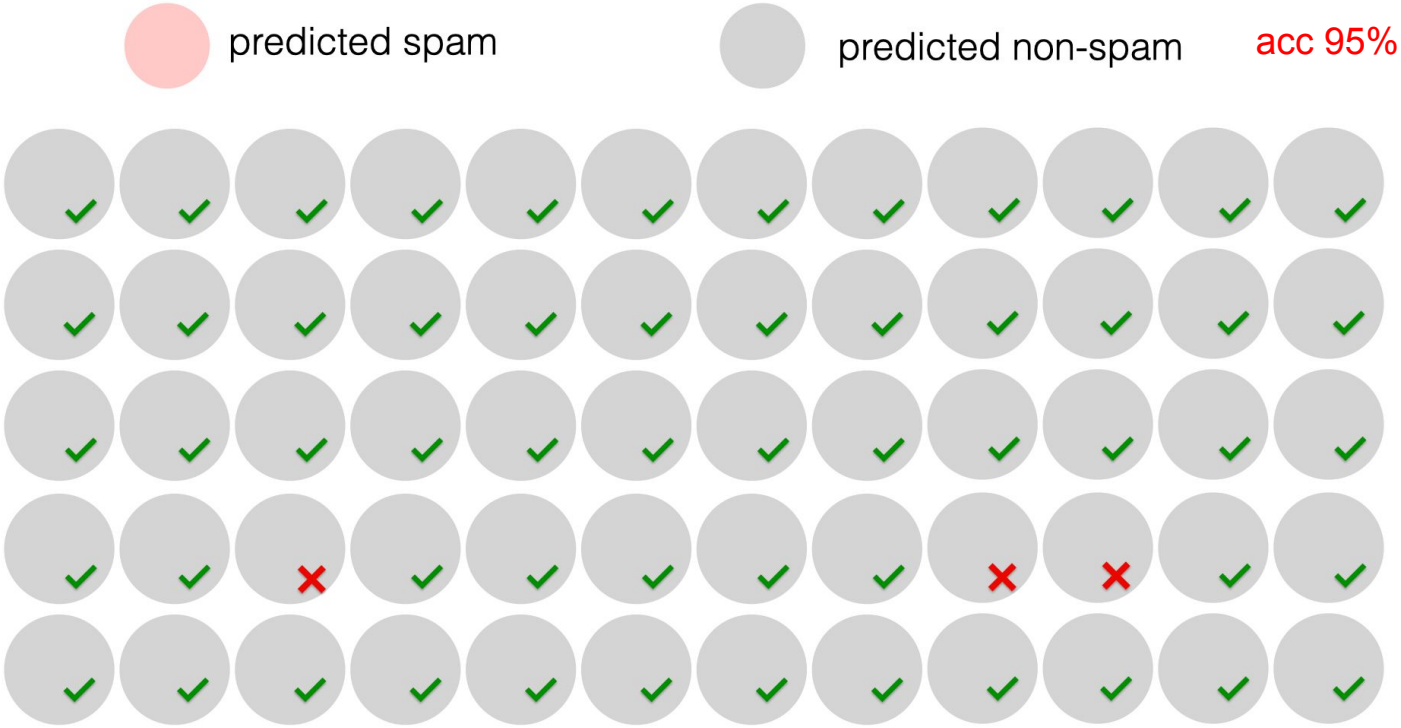
Pros:

- Accuracy is a helpful metric when you deal with **balanced classes** and care about the overall model “correctness” and not the ability to predict a specific class.
- Accuracy is **easy to explain** and communicate.

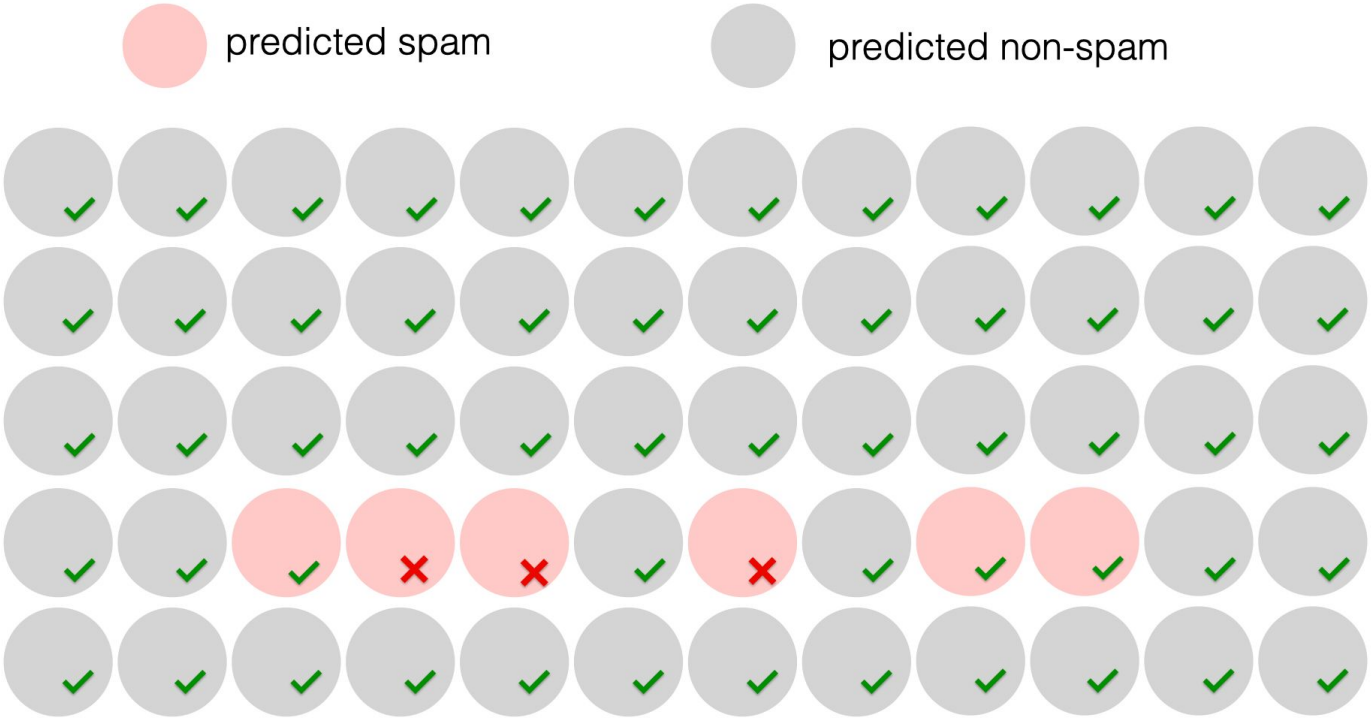
Cons:

- If you have **imbalanced classes**, accuracy is less useful since it gives equal weight to the model’s ability to predict all categories.
- Communicating accuracy in such cases **can be misleading** and disguise low performance on the target class.

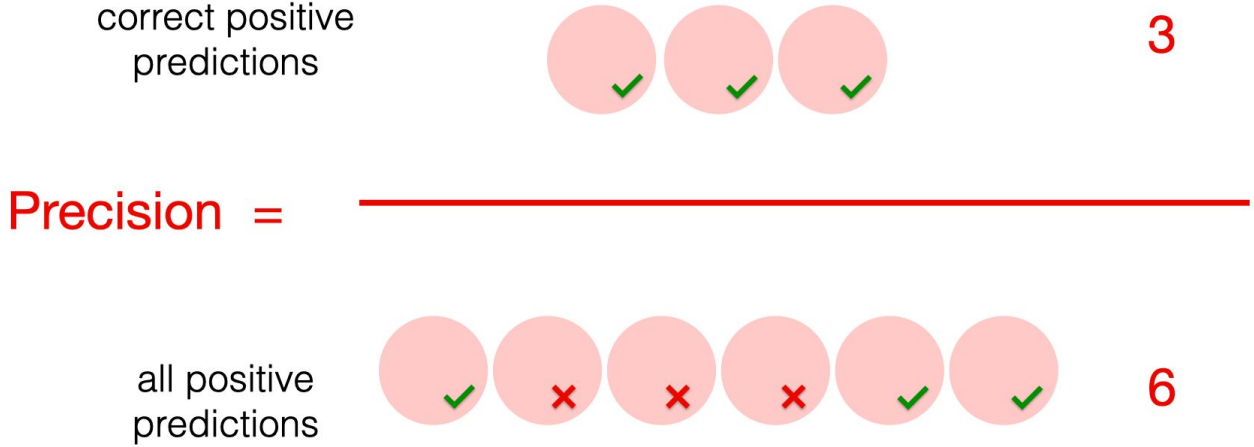
Evaluation Accuracy:



Precision



Precision 50%



Precision

Pros of the precision metric:

- It **works well** for problems with **imbalanced classes** since it shows the model correctness in **identifying the target class**.
- Precision is useful when the **cost of a false positive** is high. In this case, you typically want to be **confident in identifying the target class,** even if you miss out on some (or many) instances.

Here is the main **con of precision:**

- Precision does not consider **false negatives.** Meaning: it does not account for the cases when we miss our target event!

- Example use: email spam prediction

Recall

correct positive predictions

3

Recall =

all positive instances

3

Recall

Here are the pros of the recall metric:

- It works well for problems with **imbalanced classes** since it is focused on the model's ability to find objects of the target class.
- Recall is useful when the **cost of false negatives** is high. In this case, you typically want to find **all objects of the target class,** even if this results in some false positives (predicting a positive when it is actually negative).

Here is the **downside of recall:**

- Does not account for the cost of these **false positives**.

Example use: fraud detection.

F-score

The F1 score can be interpreted as a harmonic mean of the precision and recall, where an F1 score reaches its best value at 1 and worst score at 0.

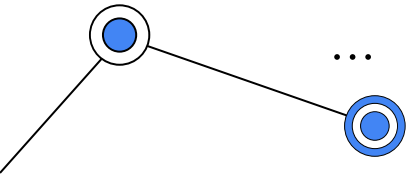
The relative contribution of precision and recall to the F1 score are equal.

$$F = 2 \cdot \frac{\mathrm{precision} \cdot \mathrm{recall}}{\mathrm{precision} + \mathrm{recall}}$$

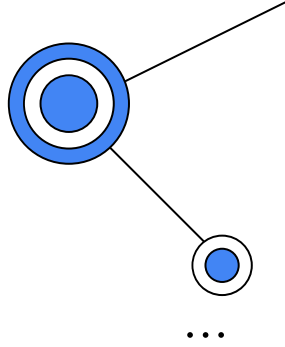
Practical Considerations

- **Mind the class balance.** Whenever you are interpreting precision, recall, and accuracy, it makes sense to evaluate the proportion of classes and remember how each metric **behaves** when dealing with **imbalanced classes.**

- **Mind the cost of error.** e.g. **Precision** is a suitable metric when you care more about **“being right”** when assigning the positive class than “detecting them all.” The **recall metric** is the **opposite**.



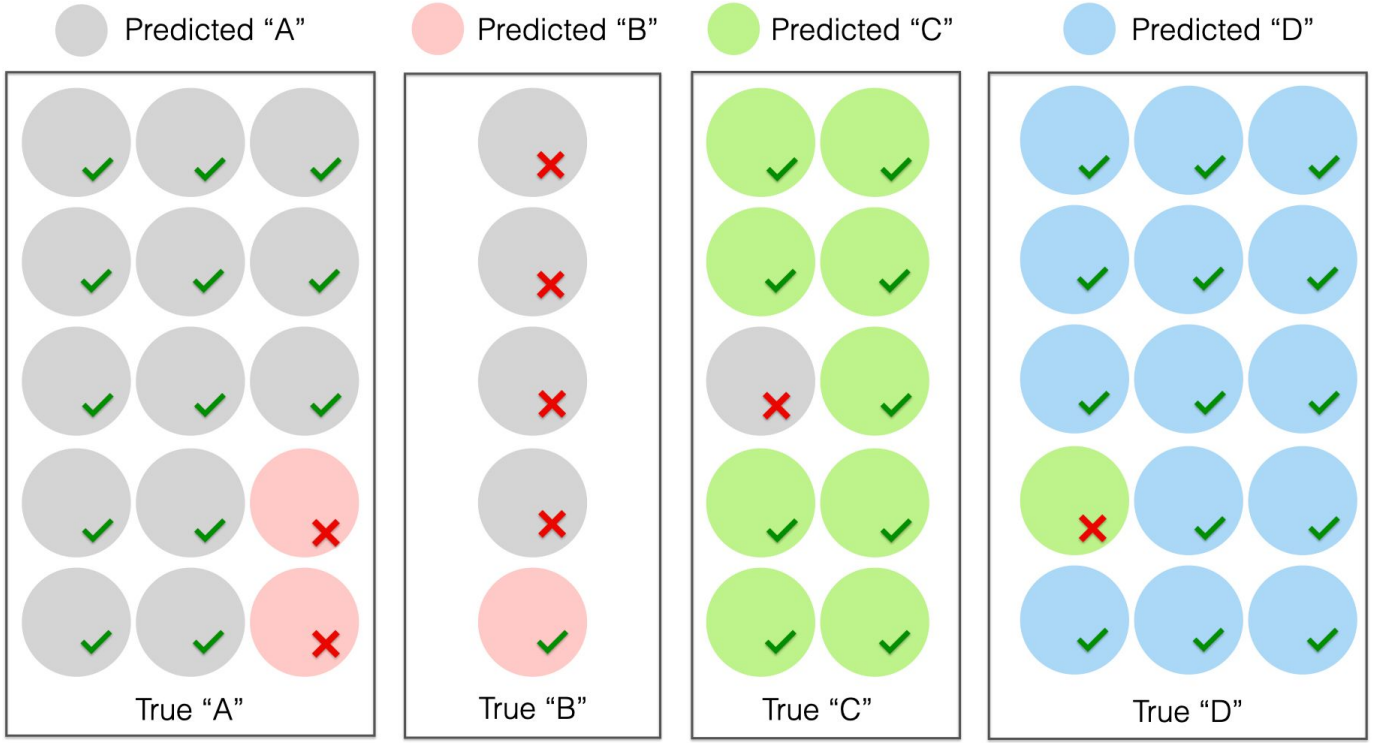
Multi-class Classification

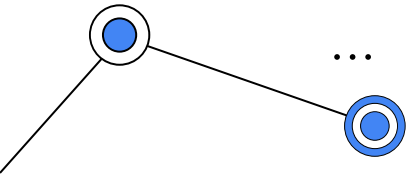


In multi-class, you have multiple categories to choose from. Some example use cases:

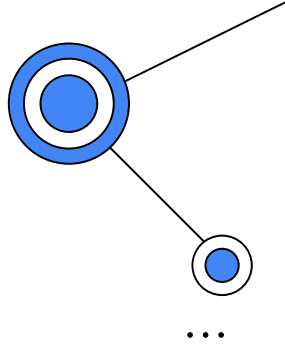
- **Product categorization.** "Clothing," "electronics," or "home and garden."
- **User segmentation**. "Small business owners," "freelancers," and "enterprise."
- **Classification of support tickets**. "Technical issue," "billing inquiry," or "product feedback."

Accuracy in multi-class





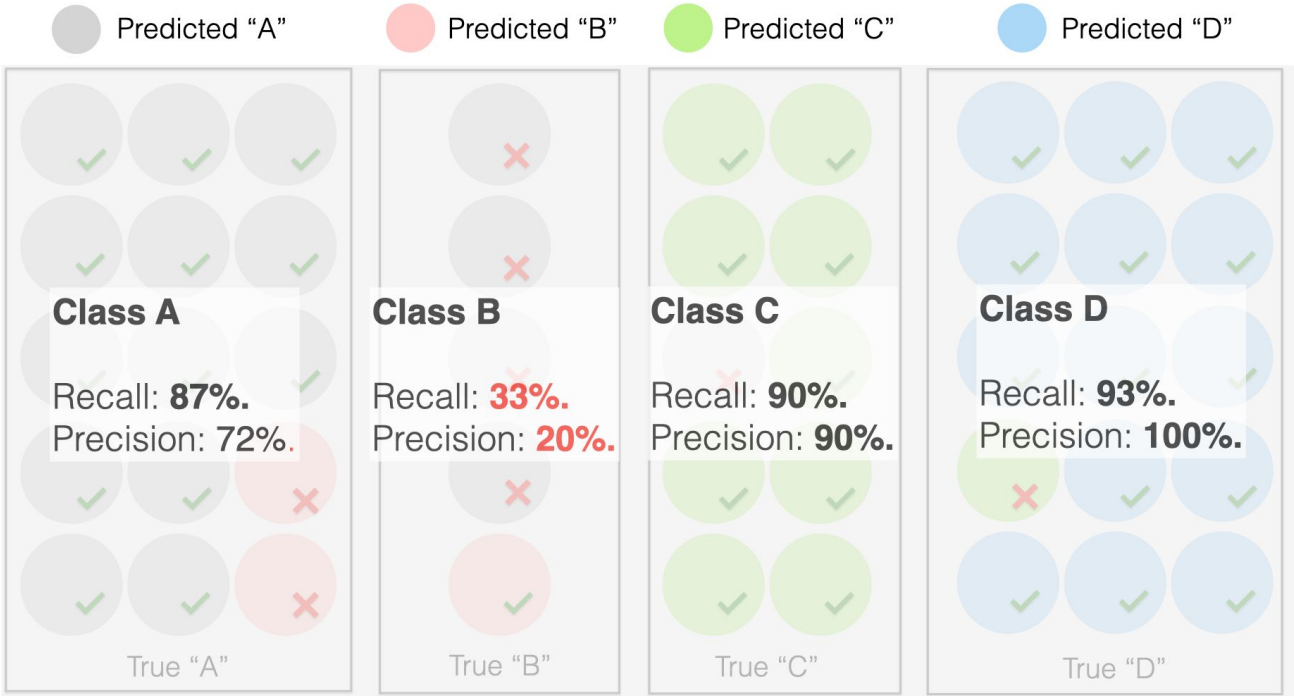
Precision and Recall in multi-class



Two main approaches to how to do that, each with its pros and cons.

- In the **first** case, you can calculate the precision and recall **for each class individually**. This way, you get multiple metrics based on the number of classes you have in the dataset.
- In the second case, you can **"average" precision and recall across all the classes** to get a single number. You can use different methods to average the precision, such as macro- or micro-averaging.

Precision and Recall in multi-class



Precision and Recall in multi-class

When is calculating precision and recall by class a good idea?

- When you have specific **classes of interest.**
- It can also be helpful when you deal with **imbalanced classes**.
- When you have a **small number of classes**.

However, there is a downside:

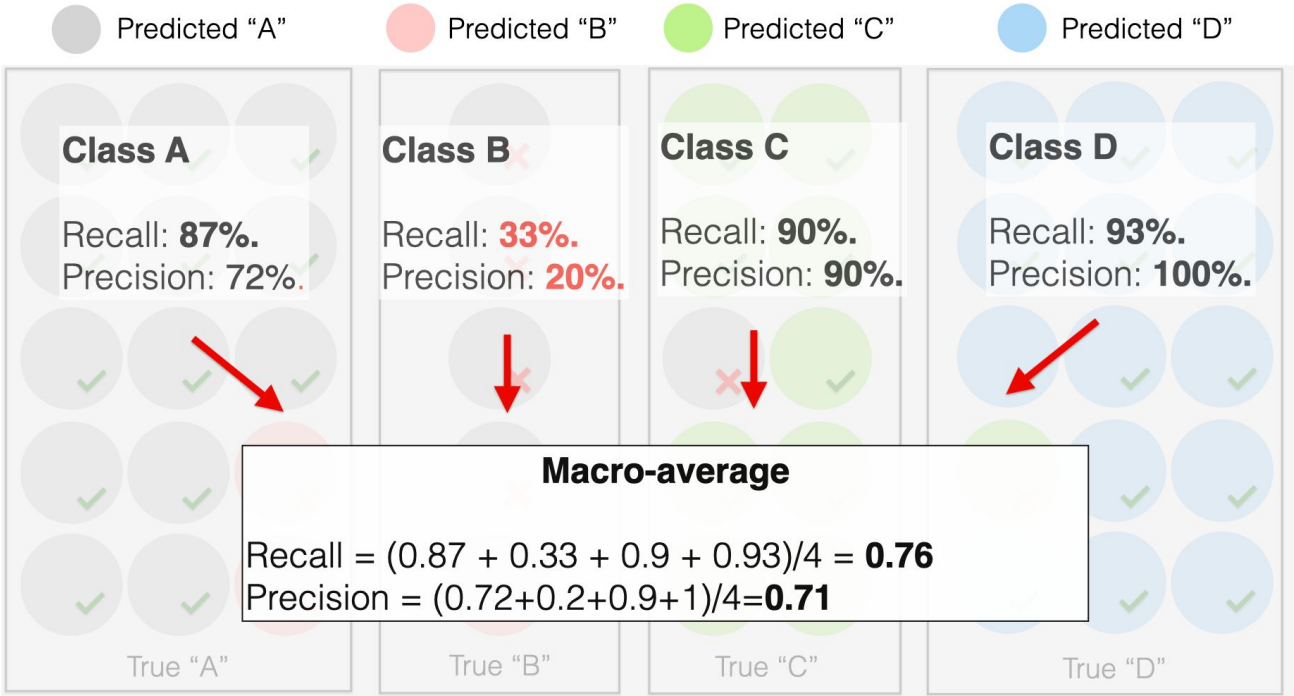
- Calculating precision and recall for each class can result in a **large number of performance metrics**. The more classes, the more metrics you get. They can be challenging to interpret and grasp simultaneously.

Averaging Precision and Recall

The idea is simple: instead of having those many metrics for every class, let's reduce it to one "**average**" metric. However, there are differences in how you can implement it.

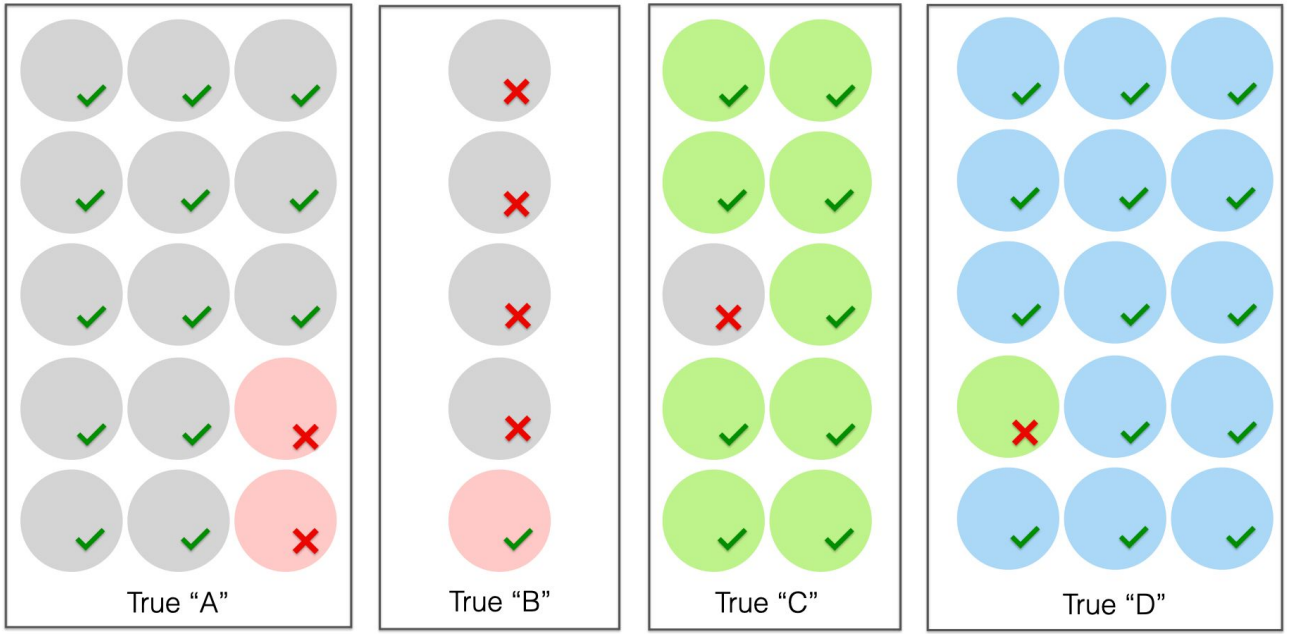
- **Macro-averaging**
- **Micro-averaging**

Macro Average



Micro Average

All predictions and true classes

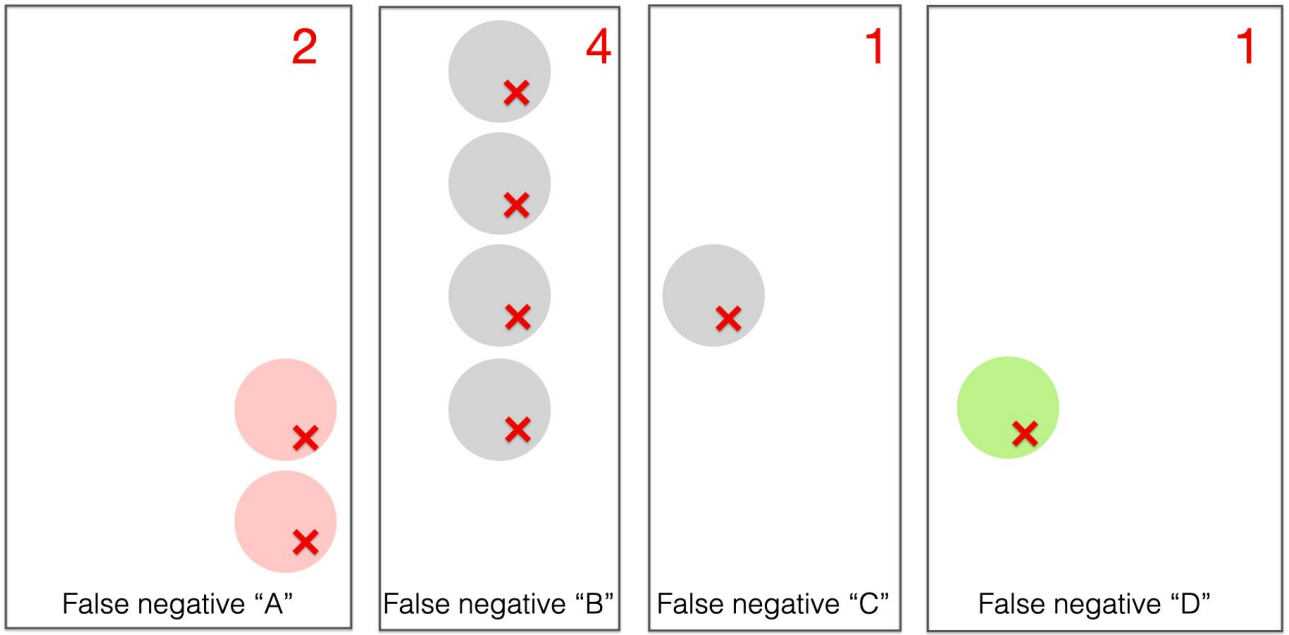


Micro Average

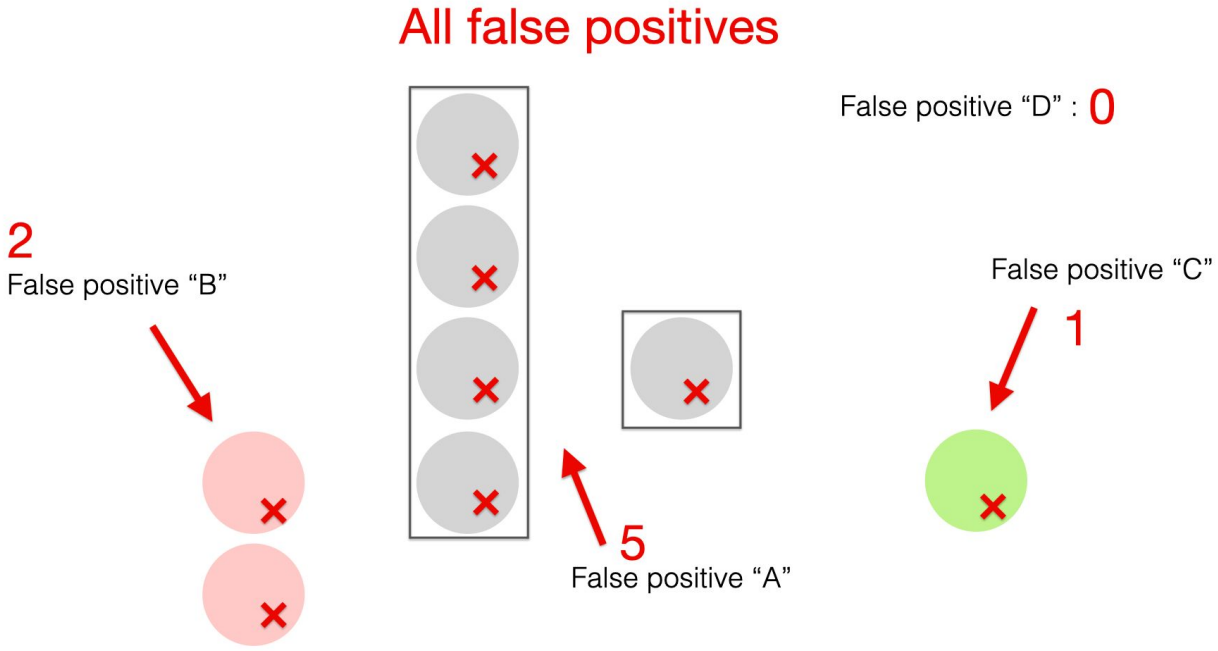
All true positives

Micro Average

All false negatives



Micro Average



Micro Average

Total TP | Total FP | Total FN

$$\text{Precision}_{\text{Micro-average}} = \frac{13 + 1 + 9 + 14}{13 + 1 + 9 + 14 + 2 + 5 + 1 + 0} = 0.82$$

$$\text{Recall}_{\text{Micro-average}} = \frac{13 + 1 + 9 + 14}{13 + 1 + 9 + 14 + 2 + 4 + 1 + 1} = 0.82$$

Macro vs. micro-average

Macro-averaging: The macro-average gives equal weight to each class, regardless of the number of instances.

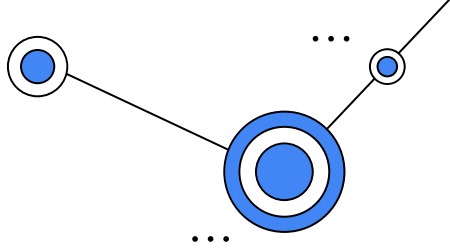
Micro-averaging: The micro-average gives equal weight to each instance, regardless of the class label and the number of cases in the class.

Example results

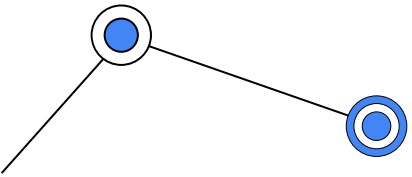
- **Macro-average precision is 76%.**
- **Macro-average recall is 71%.**
- **Micro-average precision and recall are both 82%.**

Recap

- Calculating precision and recall by class
- Macro-averaging shows average performance across classes, treating each class as equally important.
- Micro-averaging gives equal weight to every instance and shows average performance across all predictions.
- You might prefer one metric over another depending on the class balance and their relative importance.

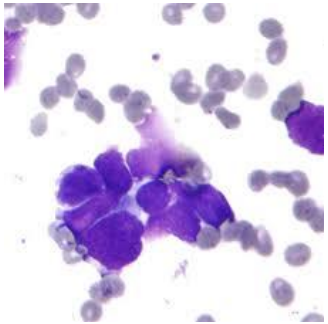


Classification Examples



Breast Cancer Wisconsin Dataset

Features are computed from a digitized image of a fine needle aspirate (FNA) of a breast mass. They describe characteristics of the cell nuclei present in the image.



Ten real-valued features are computed for each cell nucleus:

- a) radius (mean of distances from center to points on the perimeter)
  b) texture (standard deviation of gray-scale values)
  c) perimeter
  d) area
  e) smoothness (local variation in radius lengths)
  f) compactness (perimeter^2 / area - 1.0)
  g) concavity (severity of concave portions of the contour)
  h) concave points (number of concave portions of the contour)
  i) symmetry
  j) fractal dimension ("coastline approximation" - 1)

The mean, standard error and "worst" or largest (mean of the three largest values) of these features were computed for each image, resulting in 30 features.

MACHINE LEARNING ENGINEERING